SAE Base de données
Rapport d'implémentation et de validation

Projet : (MySQL)

Auteurs : Mohamed, Damien , Maxence

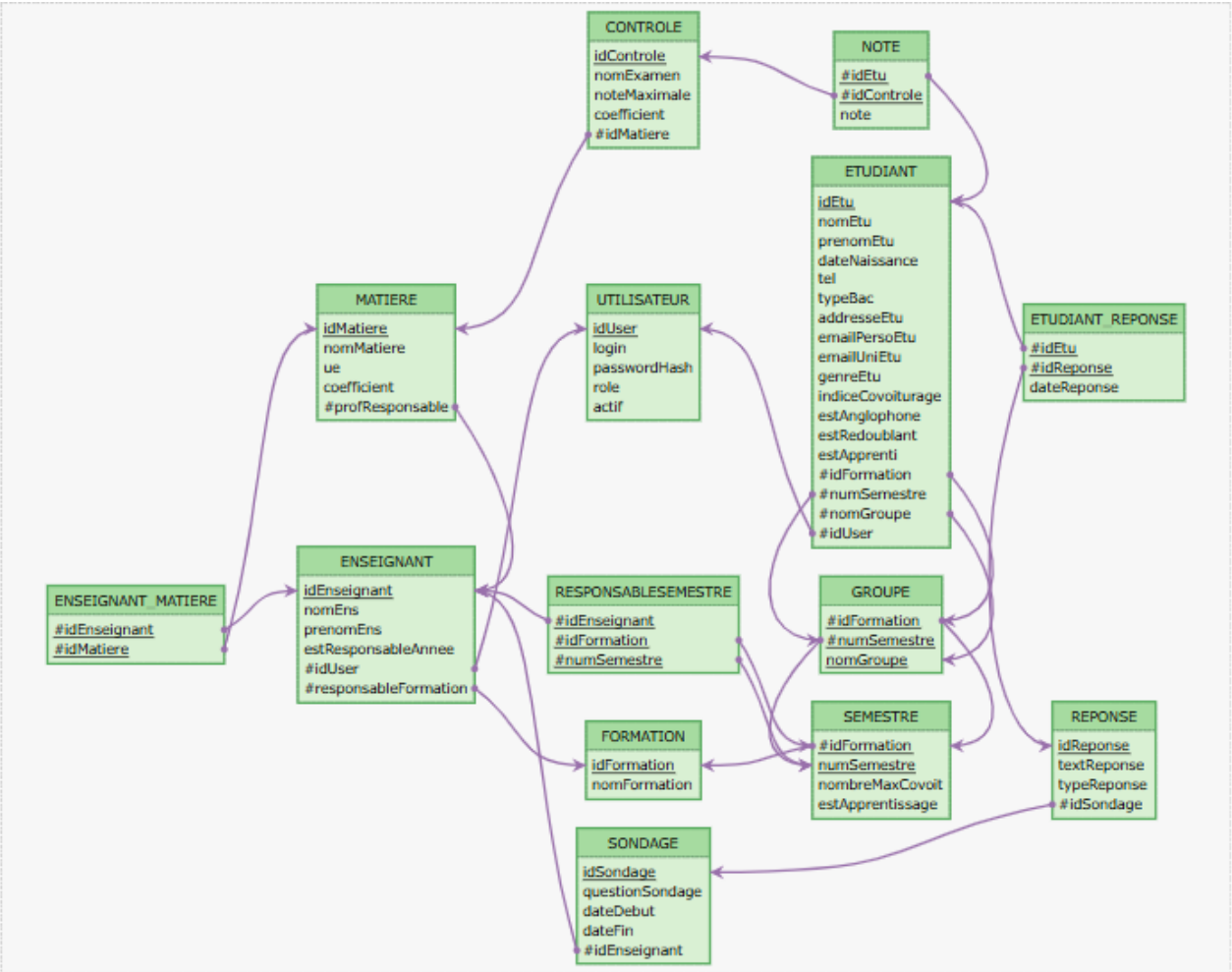
Sommaire

Sommaire 2
1. Introduction 3
3. Requêtes, vues.....6
4. Requêtes, Fonction.....9
5. Requêtes, pocedure 10
1. Requêtes, Trigger 14

1. Introduction

La base de données modélise une plateforme d'assistance à la constitution de groupes pour une promotion donnée.
Le cœur du modèle repose sur :

1. La structure pédagogique : *Formation → Semestre → Groupes*
2. Les acteurs : *Étudiants* et *Enseignants* (rattachés à un compte éventuel)
3. Les éléments d'évaluation : *Matières → Contrôles → Notes*
4. Les préférences/critères via sondages : *Sondage → Réponses → Choix des étudiants*
5. Les responsabilités : enseignants responsables de semestre et associations



2. Implémentation de la base de données (aspect statique)

La base modélise une plateforme d'aide à la constitution de groupes d'une promotion (TD/TP) à partir d'informations administratives et pédagogiques.

Une **Formation** (ex. BUT Info, BUT MMI) est découpée en Semestres(il y en a 6 semestre) . Un semestre fixe des paramètres globaux, notamment le nombre maximal de covoitureurs autorisés (contrainte fonctionnelle du sujet).

Chaque semestre possède des Groupes (unifiés TP/TD : un groupe est un "groupe pédagogique").

Les **Étudiants** appartiennent obligatoirement à une promotion (Formation + Semestre) et peuvent être affectés à un groupe (affectation optionnelle, nomGroupe peut être NULL tant que les groupes ne sont pas finalisés).

Les **Enseignants** portent les responsabilités pédagogiques : un enseignant peut être responsable de formation ou d'année, et certains sont responsables de semestre (table d'association).

Les **matières** et **contrôles** permettent de stocker des évaluations, et la table **NOTE** enregistre les notes des étudiants par contrôle (clé primaire composée, garantissant une seule note par étudiant et par contrôle).

La table **Note** modélise le résultat d'un étudiant à un contrôl**e** : elle relie un Etudiant à un Controle et stocke la **valeur de la note** obtenue.

Le covoiturage est modélisé via **indiceCovoiturage** côté étudiant (identifiant de "voiture/groupe de covoit") et la limite est définie par **Semestre.nombreMaxCovoit** : cette combinaison permet ensuite d'écrire des vues/contraintes/triggers pour vérifier qu'un même indice n'excède pas la capacité autorisée pour la promotion concernée.

indiceCovoiturage représente un identifiant commun choisi par les étudiants (même indice volonté d'être ensemble).

nombreMaxCovoit fixe la capacité maximale autorisée pour une "voiture" dans cette promotion (valeur typique 2 à 4).

La règle fonctionnelle attendue par le sujet ("limiter le nombre de covoitureurs") s'exprime naturellement par des vues de contrôle ou des triggers (ex. empêcher qu'un même indice dépasse nombreMaxCovoit pour un (idFormation,numSemestre)).

Table	Rôle (métier)	Clé primaire	Liens principaux (FK)
Formation	Référentiel des formations	idFormation	—
Semestre	Paramètres d'une promotion (S1..S6) + contraintes (covoit max, apprentissage)	(idFormation, numSemestre)	idFormation → Formation
Groupe	Groupes pédagogiques d'une promo (TP/TD unifiés)	(idFormation, numSemestre, nomGroupe)	(idFormation,numSemestre) → Semestre
Etudiant	Dossier étudiant + appartenance promo + affectation groupe	idEtu	idUser → Utilisateur, (idFormation,numSemestre) → Semestre, (idFormation,numSemestre,nomGroupe) → Groupe
Utilisateur	Authentification/autorisation (rôles)	idUser	—
Enseignant	Identité enseignant + responsabilités	idEnseignant	idUser → Utilisateur, responsableFormation → Formation
Matiere	Référentiel matières + responsable	idMatiere	profResponsable → Enseignant
Controle	Évaluations d'une matière (DS, TP noté, etc.)	idControle	idMatiere → Matiere
note	Notes : association Étudiant–Contrôle	(idEtu, idControle)	idEtu → Etudiant, idControle → Controle
Sondage	Question posée par un enseignant à une promo	idSondage	idEnseignant → Enseignant
Reponse	Réponses possibles d'un sondage	idReponse	idSondage → Sondage
ResponsableSemestre	Quels enseignants sont responsables de quelle promo	(idEnseignant,idFormation,numSemestre)	idEnseignant → Enseignant, (idFormation,numSemestre) → Semestre
Enseignant_Matiere	Qui enseigne quoi (intervenants)	(idEnseignant, idMatiere)	idEnseignant → Enseignant, idMatiere → Matiere

3. Requêtes, vues

Cette section regroupe les requêtes SQL (et vues) permettant de vérifier que l'implémentation répond aux fonctionnalités attendues par les maquettes.

- Lister les étudiants d'une formation et leurs informations de compte.

  - **Description de la vue : Vue_Bulletin_Notes**
  - **Objectif :** Cette vue est conçue pour générer automatiquement le relevé de notes de chaque étudiant. Elle calcule la **moyenne pondérée par matière**, ce qui permet d'obtenir un résultat académique fidèle à la réalité des coefficients, contrairement à une moyenne arithmétique simple.

```
CREATE OR REPLACE VIEW Vue_Bulletin_Notes AS
SELECT
   e.idEtu,
   e.numEtudiant,
   e.nomEtu,
   e.prenomEtu,
   m.idMatiere,
   m.nomMatiere,

   -- Moyenne pondérée sur les contrôles de la matière
   ROUND(
     SUM(p.note * c.coefficient) / NULLIF(SUM(c.coefficient), 0),
     2
   ) AS moyenne_matiere

FROM Etudiant e
JOIN Note  p ON p.idEtu = e.idEtu
JOIN Controle c ON c.idControle = p.idControle
JOIN Matiere  m ON m.idMatiere = c.idMatiere

GROUP BY
   e.idEtu, e.numEtudiant, e.nomEtu, e.prenomEtu,
   m.idMatiere, m.nomMatiere;
```

VUE : Liste publique d'une promotion

-- Objectif : afficher uniquement les infos "publiques" demandées dans le sujet

```
CREATE OR REPLACE VIEW v_liste_promo_publique AS
SELECT
  e.idFormation, e.numSemestre,
  e.nomEtu, e.prenomEtu, e.genreEtu, e.typeBac,
  e.emailUniEtu,
  e.nomGroupe
FROM Etudiant e;
```

VUE : Note matière par étudiant

-- Objectif : calcul intermédiaire : moyenne matière pondérée par contrôles

```
CREATE OR REPLACE VIEW v_note_matiere_etudiant AS
SELECT
  n.idEtu,
  c.idMatiere,
  SUM(n.note * c.coefficient) / NULLIF(SUM(c.coefficient), 0) AS noteMatiere
FROM Note n
JOIN Controle c ON c.idControle = n.idControle
GROUP BY n.idEtu, c.idMatiere;
```

VUE : Score global étudiant (sur l'ensemble des matières)

-- Objectif : score académique global pondéré par le coefficient des matières

```
CREATE OR REPLACE VIEW v_score_global_etudiant AS
SELECT
  v.idEtu,
  SUM(v.noteMatiere * m.coefficient) / NULLIF(SUM(m.coefficient), 0) AS scoreGlobal
FROM v_note_matiere_etudiant v
JOIN Matiere m ON m.idMatiere = v.idMatiere
GROUP BY v.idEtu;
```

VUE : Qualité des groupes (moyenne + écart-type des scores)

-- Objectif : tableau de bord pour vérifier l'homogénéité des groupes

```
CREATE OR REPLACE VIEW v_qualite_groupes AS
SELECT
  e.idFormation, e.numSemestre, e.nomGroupe,
  COUNT(*) AS effectif,
  AVG(s.scoreGlobal) AS moyenneGroupe,
  STDDEV_POP(s.scoreGlobal) AS ecartTypeGroupe
FROM Etudiant e
JOIN v_score_global_etudiant s ON s.idEtu = e.idEtu
WHERE e.nomGroupe IS NOT NULL
GROUP BY e.idFormation, e.numSemestre, e.nomGroupe;
```

VUE : Dossier pédagogique (détail des notes par contrôle)

Objectif : "fiche étudiant" détaillée (matière, examen, note, prof responsable)

```
CREATE OR REPLACE VIEW Vue_Dossier_Pedagogique AS
SELECT
    e.idEtu,
    e.nomEtu,
    e.prenomEtu,
    m.nomMatiere,
    c.nomExamen,
    n.note,
    c.noteMaximale,
    c.coefficient AS Coeff_Examen,
    ens.nomEns AS Enseignant_Responsable
FROM Etudiant e
JOIN Note n        ON n.idEtu = e.idEtu
JOIN Controle c    ON c.idControle = n.idControle
JOIN Matiere m     ON m.idMatiere = c.idMatiere
JOIN Enseignant ens ON ens.idEnseignant = m.profResponsable;
```

4. Requêtes, Fonction

6) Moyenne globale simple d'un étudiant (sur tous les contrôles)

```
CREATE OR REPLACE FUNCTION MoyenneEtudiantFn(p_idEtu INT)
RETURNS DECIMAL(5,2)
BEGIN
   DECLARE v_moy DECIMAL(5,2);

   SELECT
      ROUND(
         SUM(p.note * c.coefficient) / SUM(c.coefficient),
         2
      )
   INTO v_moy
   FROM Note p
   JOIN Controle c ON p.idControle = c.idControle
   WHERE p.idEtu = p_idEtu;

   RETURN v_moy;
END//

DELIMITER ;
```

Cette fonction permet de normaliser les résultats en convertissant toute note brute sur une base de 20, tout en incluant une sécurité qui évite les erreurs de division par zéro si aucune note maximale n'est définie.

```
DELIMITER //

CREATE OR REPLACE FUNCTION note_sur_20(p_note DECIMAL(6,2), p_noteMax DECIMAL(6,2))
RETURNS DECIMAL(6,2)
BEGIN
  IF p_noteMax IS NULL OR p_noteMax = 0 THEN
    RETURN NULL;
  END IF;
  RETURN ROUND((p_note * 20) / p_noteMax, 2);
END //
DELIMITER ;
```

5. Requêtes, pocedure

Afin d'illustrer la dynamique de notre base de données, nous avons sélectionné des procédures qui représentent des fonctionnalités de notre application

affecter_etudiant_groupe (Procédure)

Cette procédure assigne un étudiant à un groupe de TD ou TP (ex: Groupe A). Elles vérifient au préalable que le groupe existe bien dans la formation et le semestre de l'étudiant pour éviter les erreurs d'affectation.

```
DELIMITER //
CREATE OR REPLACE PROCEDURE AffecterGroupe(
    IN p_idEtu INT,
    IN p_nomGroupe VARCHAR(50)
)  BEGIN
    DECLARE v_idFormation INT;
    DECLARE v_numSemestre INT;
     SELECT idFormation, numSemestre
    INTO v_idFormation, v_numSemestre
    FROM Etudiant
```

```
  WHERE idEtu = p_idEtu;
  UPDATE Etudiant
  SET nomGroupe = p_nomGroupe
  WHERE idEtu = p_idEtu
    AND EXISTS (
      SELECT 1
        FROM Groupe g
        WHERE g.idFormation = v_idFormation
          AND g.numSemestre = v_numSemestre
        AND g.nomGroupe = p_nomGroupe
    );
END//
DELIMITER ;
```

sp_creer_sondage (Procédure)

Elle permet de créer un sondage complet et ses choix de réponses (séparés par des |) en une seule commande. Son but est de simplifier la création de questionnaires pour les enseignants sans avoir à faire de multiples insertions manuelles.

```
  DELIMITER $$
CREATE OR REPLACE PROCEDURE sp_creer_sondage(
  IN p_idEnseignant INT,
  IN p_question VARCHAR(5000),
  IN p_dateFin DATETIME,
  IN p_typeReponse VARCHAR(50),
  IN p_reponses TEXT
)
BEGIN
  DECLARE v_idSondage INT;
  DECLARE v_rest TEXT;
  DECLARE v_pos INT;
  DECLARE v_item VARCHAR(1000);
 INSERT INTO Sondage(questionSondage, dateFin, idEnseignant)
 VALUES (p_question, p_dateFin, p_idEnseignant);
SET v_idSondage = LAST_INSERT_ID();
SET v_rest = p_reponses;
```

```
WHILE v_rest IS NOT NULL AND LENGTH(v_rest) > 0 DO
SET v_pos = LOCATE('|', v_rest);
IF v_pos = 0 THEN SET v_item =TRIM(v_rest);
      SET v_rest = '';
    ELSE
      SET v_item = TRIM(SUBSTRING(v_rest, 1, v_pos - 1));
      SET v_rest = SUBSTRING(v_rest, v_pos + 1);
    END IF;
  IF v_item <> '' THEN
      INSERT INTO Reponse(textReponse, typeReponse, idSondage)
      VALUES (v_item, p_typeReponse, v_idSondage);
    END IF;
  END WHILE;

  SELECT v_idSondage AS idSondage_cree;
END $$

DELIMITER ;
```

ajouter_note

Elle permet d'insérer ou de mettre à jour une note tout en vérifiant qu'elle ne dépasse pas la note maximale de l'examen. Elle garantit l'intégrité des données en empêchant la saisie de notes invalides Procédure.

```
DELIMITER $$
CREATE OR REPLACE PROCEDURE ajouter_note(IN p_idEtu INT, IN p_idControle INT, IN p_note DECIMAL(4,2))
BEGIN
  DECLARE v_noteMax TINYINT;
  SELECT noteMaximale INTO v_noteMax FROM Controle WHERE idControle = p_idControle;
  IF v_noteMax IS NULL THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erreur : Controle inexistant.';
  END IF;
  IF p_note < 0 OR p_note > v_noteMax THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erreur : Note hors limites.';
  END IF;
  INSERT INTO Note(idEtu, idControle, note) VALUES (p_idEtu, p_idControle, p_note)
```

```
  ON DUPLICATE KEY UPDATE note = p_note;
END $$
DELIMITER ;
```

sp_modifier_etudiant_pedago

Objectif : Cette procédure permet de mettre à jour le **profil pédagogique et social** d'un étudiant existant sans toucher à ses données d'identité (Nom, Prénom, Email).

Fonctionnement en 2 étapes :

1. **Sécurité :** Elle vérifie d'abord si l'étudiant existe grâce à son idEtu. Si ce n'est pas le cas, elle bloque l'opération et renvoie un message d'erreur ("Etudiant introuvable").

```
DELIMITER //


CREATE OR REPLACE PROCEDURE sp_modifier_etudiant_pedago(
  IN p_idEtu INT,
  IN p_typeBac VARCHAR(50),
  IN p_estRedoublant BIT,
  IN p_estApprenti BIT,
  IN p_estAnglophone BIT,
  IN p_indiceCovoit INT
)
BEGIN
  -- Vérification de l'existence de l'étudiant
  IF NOT EXISTS (SELECT 1 FROM Etudiant WHERE idEtu=p_idEtu) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='Etudiant introuvable';
  END IF;


  -- Mise à jour des infos pédagogiques
  UPDATE Etudiant
  SET typeBac = p_typeBac,
      estRedoublant = p_estRedoublant,
      estApprenti = p_estApprenti,
      estAnglophone = p_estAnglophone,
```

```
    indiceCovoiturage = p_indiceCovoit
  WHERE idEtu = p_idEtu;
END//


DELIMITER ;
```

1. Requêtes, Trigger

Les triggers garantissent la cohérence des données en temps réel en bloquant toute action qui ne respecte pas les règles de gestion du département.

1. Vérification de la cohérence des rôles (Utilisateurs)

Objectif : Empêcher qu'un compte utilisateur ayant un rôle "Enseignant" soit lié à une fiche "Étudiant", et inversement

```
CREATE OR REPLACE TRIGGER trg_verif_role_etudiant
BEFORE INSERT ON Etudiant FOR EACH ROW
BEGIN
  IF NEW.idUser IS NOT NULL THEN
    IF (SELECT role FROM Utilisateur WHERE idUser = NEW.idUser) <> 'ETUDIANT' THEN
      SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Le compte doit avoir le rôle ETUDIANT';
    END IF;
  END IF;
END$$


-- Trigger pour Enseignant
CREATE OR REPLACE TRIGGER trg_verif_role_enseignant
BEFORE INSERT ON Enseignant FOR EACH ROW
BEGIN
  IF NEW.idUser IS NOT NULL THEN
    IF (SELECT role FROM Utilisateur WHERE idUser = NEW.idUser) <> 'ENSEIGNANT' THEN
      SIGNAL SQLSTATE '45001' SET MESSAGE_TEXT = 'Le compte doit avoir le rôle ENSEIGNANT';
    END IF;
  END IF;
END$$
```

2. Sécurité des Notes

Objectif : Interdire la saisie de notes supérieures au barème défini pour un contrôle.

```
 CREATE OR REPLACE TRIGGER trg_verif_note_max

BEFORE INSERT ON Note FOR EACH ROW

BEGIN

  DECLARE maxNote INT;

  SELECT noteMaximale INTO maxNote FROM Controle WHERE idControle = NEW.idControle;

  IF NEW.note > maxNote THEN

    SIGNAL SQLSTATE '45002' SET MESSAGE_TEXT = 'La note dépasse la note maximale du contrôle';

  END IF;

END$$
```

3. Gestion des Sondages (Dates et Unicité)

Objectif : Garantir qu'un sondage est actif au moment du vote et qu'un étudiant ne peut pas voter deux fois si le choix est unique ("bouton").

```
CREATE OR REPLACE TRIGGER trg_verif_note_max

BEFORE INSERT ON Note FOR EACH ROW

BEGIN

  DECLARE maxNote INT;

  SELECT noteMaximale INTO maxNote FROM Controle WHERE idControle = NEW.idControle;

  IF NEW.note > maxNote THEN

    SIGNAL SQLSTATE '45002' SET MESSAGE_TEXT = 'La note dépasse la note maximale du contrôle';

  END IF;

END$$
```

6. Conclusion

Pour conclure, ce projet nous a permis de concevoir et de mettre en œuvre un système de gestion pédagogique complet, intégrant à la fois le suivi académique (notes, moyennes) et des fonctionnalités de vie étudiante (sondages, covoiturage). L'architecture de la base de données a été pensée pour garantir une intégrité maximale des données grâce à une hiérarchie stricte entre les formations, les semestres et les groupes.

Ce projet a été une opportunité majeure pour renforcer nos compétences en SQL et en modélisation de données. Nous avons réussi à transformer un cahier des charges complexe en une solution technique viable, capable de servir de fondation solide à une future interface utilisateur (PHP/Java).